

5CM506 Data driven systems

Introduction to Normalisation

Dr. Maqbool Hussain

(m.Hussain@derby.ac.uk)

Aims of the Session

This session aims to help you:

1. Understand the main concepts behind different types of E/R diagrams
2. Explain the steps to produce a normalised database
3. Describe the different types of anomalies in the creation of a relational database

Overview

- **Recap: Conceptual & Logical E/R Diagram**
- Physical E/R Diagram
- Normal Forms: 1st, 2nd and 3rd
- Data Redundancy and Update Anomalies

Example

- Derbyshire Deliveries take orders from a wide range of clients. Each client has a reference number, name and address. They also have a credit rating and a credit limit
- A client may place any number of orders. When an order is placed, we give it an order number and a date and associate it with a set of stock items
- Stock items have a stock number, description, level and price
- When we order an item, we record the quantity and sale price of the item ordered

Conceptual E/R Diagram

A conceptual E/R diagram is independent of any particular database, or even type of database system. In detail:

- **Entities** always shown
- **Attributes** may or may not be shown
- **Relationships** always shown, but foreign key are not shown

Conceptual E/R Diagram

A conceptual E/R diagram is independent of any particular database, or even type of database system. In detail:

- **Entities** always shown
- **Attributes** may or may not be shown
- **Relationships** always shown, but foreign key are not shown

Many-to-many relationships are allowed

Multi-valued attributes are allowed (e.g. multiple phone numbers in a *PhoneNumbers* attribute)



Conceptual E/R Diagram

CLIENTS

RefNumber

Name

Address

CreditRating

CreditLimit

STOCK

StockNumber

Description

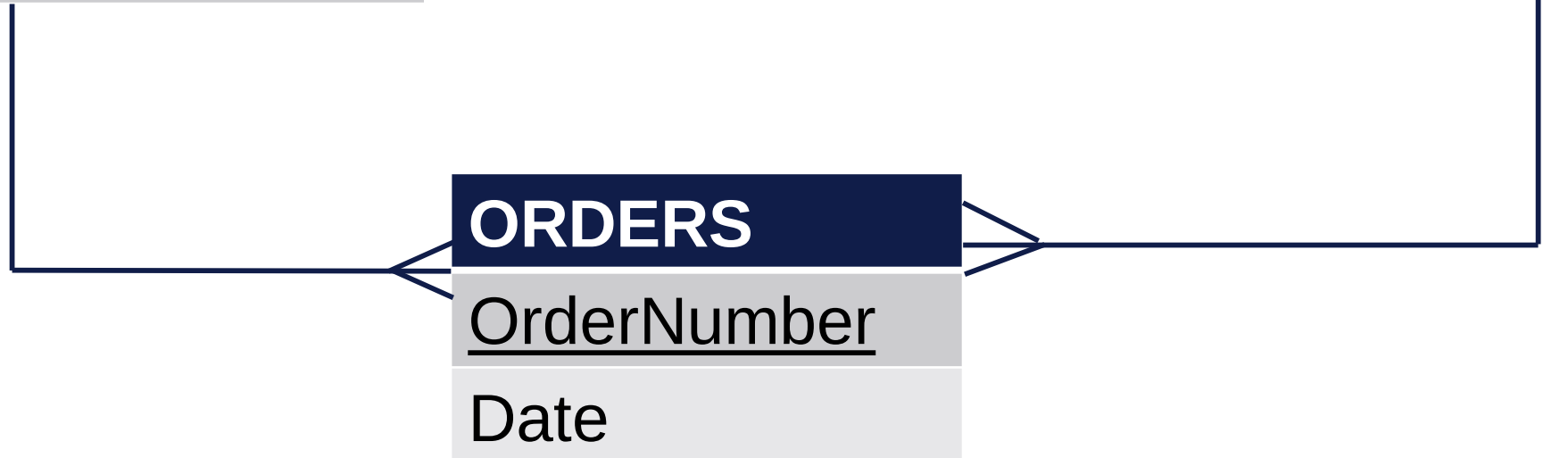
Level

Price

ORDERS

OrderNumber

Date



Logical E/R Diagram

A logical E/R diagram is intended for a relational/SQL database. In detail:

- **Entities** always shown
- **Attributes** always shown
- **Relationships** always shown, and foreign key are clearly indicated

Logical E/R Diagram

A logical E/R diagram is intended for a relational/SQL database. In detail:

- **Entities** always shown
- **Attributes** always shown
- **Relationships** always shown, and foreign key are clearly indicated

Many-to-many relationships are NOT allowed

Multi-valued attributes are NOT allowed



Logical E/R Diagram

CLIENTS

RefNumber

Name

Address

CreditRating

CreditLimit

ORDERS

OrderNumber

Date

ReferenceNumber (fk)

STOCK

StockNumber

Description

Level

Price

ORDER_DETAIL

StockNumber (fk)

OrderNumber (fk)

Quantity

SalePrice

Overview

- Recap: Conceptual & Logical E/R Diagram
- **Physical E/R Diagram**
- Normal Forms: 1st, 2nd and 3rd
- Data Redundancy and Update Anomalies

Physical E/R Diagram

Conceptual E/R Diagram

- Shows ***what*** the database should contain, but does not specify ***how***

Logical E/R Diagram

- Shows ***how*** the database should be implemented within a relational DBMS, but does not ***specify*** which relational DBMS

Physical E/R Diagram

Conceptual E/R Diagram

- Shows ***what*** the database should contain, but does not specify ***how***

Logical E/R Diagram

- Shows ***how*** the database should be implemented within a relational DBMS, but does not ***specify*** which relational DBMS

Physical E/R Diagram

- Shows ***how*** the database should be implemented for a ***specific*** relational DBMS

Physical E/R Diagram

A physical E/R diagram is intended for a specific relational/SQL DBMS. In addition to what the logical E/R diagram shows, we may have:

- Attribute types
- Product-specific information

Physical E/R Diagram

A physical E/R diagram is intended for a specific relational/SQL DBMS. In addition to what the logical E/R diagram shows, we may have:

- Attribute types
- Product-specific information

An experienced developer can convert a conceptual E/R diagram directly to a physical E/R diagram consisting of table definitions for a specific DBMS

For complex projects, this might not be the case anymore, and the need of logical E/R diagrams arises

Physical E/R Diagram

To convert a logical E/R model into a physical E/R diagram there are a certain number of steps we want to follow:

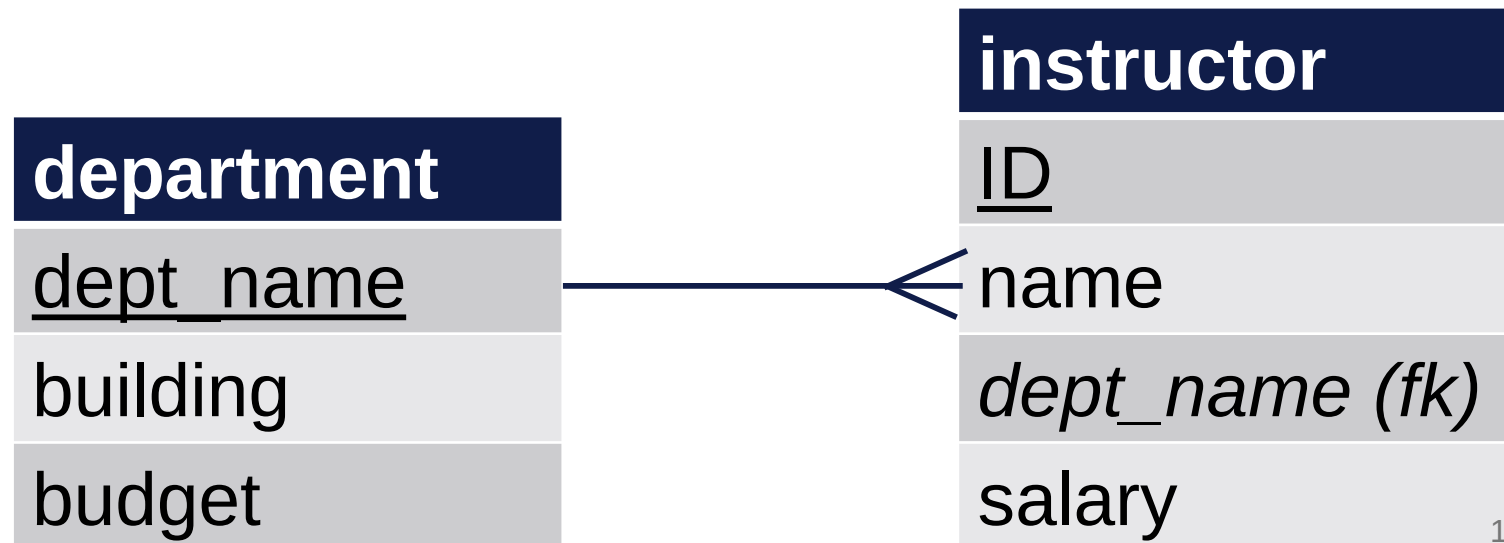
1. For each entity, create a table and put one column for each attribute of the entity.
2. When two entities participate in a 1-M relationship, insert a foreign key column into the table representing the M entity

Physical E/R Diagram

To convert a logical E/R model into a physical E/R diagram there are a certain number of steps we want to follow:

1. For each entity, create a table and put one column for each attribute of the entity.
2. When two entities participate in a 1-M relationship, insert a foreign key column into the table representing the M entity

Example: *instructor* (ID, name, dept_name fk, salary)



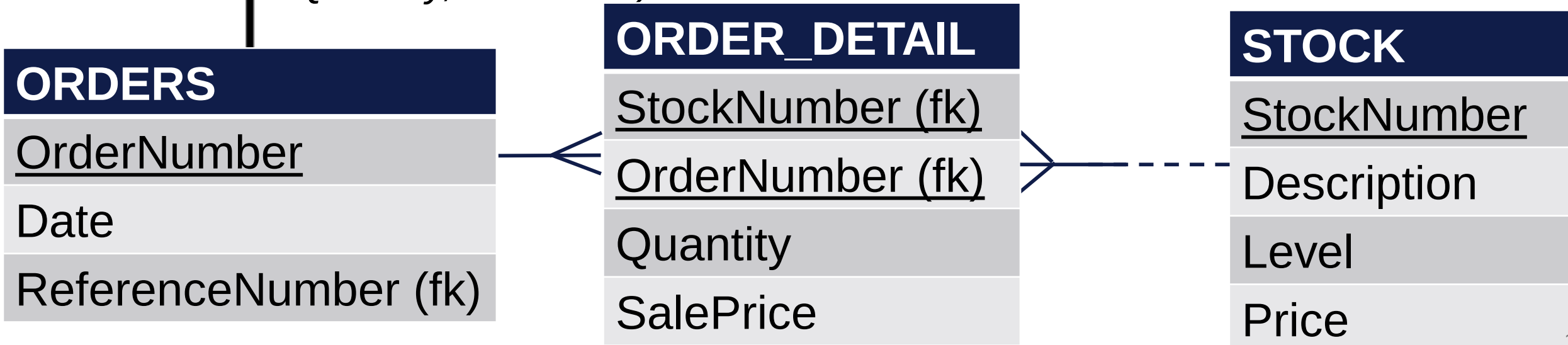
Physical E/R Diagram

3. When two entities participate in a 1-1 relationship, insert a foreign key column into one of the tables
4. When two entities participate in a M-N relationship, create a new weak entity table to represent the relationship. Its columns will consist of at least foreign keys of the participating entities table

Physical E/R Diagram

- When two entities participate in a 1-1 relationship, insert a foreign key column into one of the tables
- When two entities participate in a M-N relationship, create a new weak entity table to represent the relationship. Its columns will consist of at least foreign keys of the participating entities table

Example: *ORDER_DETAIL*(*StockNumber* *fk*, *OrderNumber* *fk*, *Quantity*, *SalePrice*)



Physical E/R Diagram

5. When two or more entities participate in a relationship, create a new table consisting of the foreign keys of the participating entities
6. In the presence of a multi-valued attribute, create a separate table with a foreign key column referencing the original entity and a column for the multi-valued attribute. Together, they form a composite primary key

Example: *StudentPhones*(*StudentNumber* *fk*, *PhoneNumber*)

Physical E/R Diagram

7. When an entity is a subclass, create tables for each subtype consisting of the primary key and the distinctive attributes. A table for the superclass consisting of the primary key and common attributes must be present

Example: *Staff*(*StaffID*, *Name*, *Phone*)

Lecturer(*StaffID* *fk*, *Degree*, *Office*), *Tutor*(*StaffID* *fk*, *Certificate*)

Overview

- Recap: Conceptual & Logical E/R Diagram
- Physical E/R Diagram
- **Normal Forms: 1st, 2nd and 3rd**
- Data Redundancy and Update Anomalies

Normalisation/Normalization

Database normalization, or simply normalization, is the process of **organising the attributes and tables** of a relational database to **reduce data redundancy and improve data integrity** (Wikipedia)

Normalisation/Normalization

Database normalization, or simply normalization, is the process of organising the attributes and tables of a relational database to reduce data redundancy and improve data integrity (Wikipedia)

The goal of relational database design is to generate a set of relation schemas that allows us

- To store information without unnecessary redundancy
- To retrieve information easily

This is accomplished by designing schemas that are in a given **normal form**

Normalisation/Normalization

Database normalization, or simply normalization, is the process of organising the attributes and tables of a relational database to reduce data redundancy and improve data integrity (Wikipedia)

The goal of relational database design is to generate a set of relation schemas that allows us

- To store information without unnecessary redundancy
- To retrieve information easily

This is accomplished by designing schemas that are in a given **normal form**

We will look at normal forms, up to 3rd normal form, as higher forms may reduce update anomalies but introduce other undesirable problems

First Normal Form (1NF)

A domain is **atomic** if elements of the domain are considered to be indivisible units

We say that a relation schema R is in **1NF** if the domains of all attributes of R are atomic

Example:

StudentID	PhoneNo
100102030	555321, 555322
100102031	555456, 555457
100102032	555115

First Normal Form (1NF)

A domain is **atomic** if elements of the domain are considered to be indivisible units

We say that a relation schema R is in **1NF** if the domains of all attributes of R are atomic

Example:

StudentID	PhoneNo
100102030	555321, 555322
100102031	555456, 555457
100102032	555115



StudentID	PhoneNo
100102030	555321
100102030	555322
100102031	555456
100102031	555457
100102032	555115

First Normal Form (1NF)

A domain is **atomic** if elements of the domain are considered to be indivisible units

We say that a relation schema R is in **1NF** if the domains of all attributes of R are atomic

Example:

StudentID	PhoneNo
100102030	555321, 555322
100102031	555456, 555457
100102032	555115



StudentID	PhoneNo
100102030	555321
100102030	555322
100102031	555456
100102031	555457
100102032	555115

First Normal Form (1NF)

A domain is **atomic** if elements of the domain are considered to be indivisible units

We say that a relation schema R is in **1NF** if the domains of all attributes of R are atomic

Example:

StudentID	PhoneNo
100102030	555321, 555322
100102031	555456, 555457
100102032	555115



StudentID	PhoneNo
100102030	555321
100102030	555322
100102031	555456
100102031	555457
100102032	555115

First Normal Form (1NF)

A domain is **atomic** if elements of the domain are considered to be indivisible units

We say that a relation schema R is in **1NF** if the domains of all attributes of R are atomic

Example:

StudentID	PhoneNo
100102030	555321, 555322
100102031	555456, 555457
100102032	555115



StudentID	PhoneNo
100102030	555321
100102030	555322
100102031	555456
100102031	555457
100102032	555115

Second Normal Form (2NF)

We say that a relation schema R is in **2NF** if it is in 1NF and all non-key attributes are fully **functionally dependent** on the primary key

Second Normal Form (2NF)

We say that a relation schema R is in **2NF** if it is in 1NF and all non-key attributes are fully **functionally dependent** on the primary key

Functional Dependence: Given an attribute (or a set of attributes) X , we say that another attribute A is functionally dependent on X , if for every X value, there is one and only one corresponding A value

$$X \rightarrow A$$

Functional dependencies are determined via common sense, knowledge about the given scenario and real-world data

Second Normal Form (2NF)

Given an attribute (or a set of attributes) X , we say that another attribute A is functionally dependent on X ($X \rightarrow A$), if for every X value, there is one and only one corresponding A value

Example:

$Q \rightarrow P ?$

$P \rightarrow Q ?$

$Z \rightarrow P ?$

$Z \rightarrow Q ?$

<u>Z</u>	P	<u>Q</u>
123	AAA	xxx
132	AAB	xyy
1108	ACA	xyz
1189	ABD	xyz
1239	ADE	xzz
1239	ADE	xzy
1566	DDE	zzx
1598	DDE	zzy

Second Normal Form (2NF)

Given an attribute (or a set of attributes) X , we say that another attribute A is functionally dependent on X ($X \rightarrow A$), if for every X value, there is one and only one corresponding A value

Example:

$Q \rightarrow P ?$ NO

$P \rightarrow Q ?$ NO

$Z \rightarrow P ?$ YES

$Z \rightarrow Q ?$ NO

<u>Z</u>	P	<u>Q</u>
123	AAA	xxx
132	AAB	xyy
1108	ACA	xyz
1189	ABD	xyz
1239	ADE	xzz
1239	ADE	xzy
1566	DDE	zzx
1598	DDE	zzy

Second Normal Form (2NF)

We need to worry about **2NF** only when there is a composite key

A table in **1NF** with a key consisting of a single attribute is inherently in **2NF**

Second Normal Form (2NF)

We need to worry about **2NF** only when there is a composite key

A table in **1NF** with a key consisting of a single attribute is inherently in **2NF**

When a non-key attribute is functionally dependent on a subset of the attributes composing the primary key, then we say that this attribute is partially dependent and we need to:

- Create a new table whose primary key consists of the attributes of the original table where the partial dependency exists plus the non-key attribute
- Remove the non-key attribute that had a partial dependence from the original table

Second Normal Form (2NF) - Example

Consider the following table;

STUD_NO	COURSE_NO	COURSE_FEE
1	C1	1000
2	C2	1500
1	C4	2000
4	C3	1000
4	C1	1000
2	C5	2000

COURSE_FEE cannot alone decide the value of COURSE_NO or STUD_NO;

COURSE_FEE together with STUD_NO cannot decide the value of COURSE_NO;

COURSE_FEE together with COURSE_NO cannot decide the value of STUD_NO;

Hence,

COURSE_FEE would be a non-prime attribute,

But, COURSE_NO $\rightarrow$ COURSE_FEE, i.e., COURSE_FEE is dependent on COURSE_NO, which is a proper subset of the candidate key.

Non-prime attribute COURSE_FEE is dependent on a proper subset of the candidate key, which is a partial dependency and so this relation is not in 2NF.

Second Normal Form (2NF) - Example

Consider the following table;

STUD_NO	COURSE_NO	COURSE_FEE
1	C1	1000
2	C2	1500
1	C4	2000
4	C3	1000
4	C1	1000
2	C5	2000

To convert the above relation to 2NF,
we need to split the table into two tables such as :
Table 1: STUD_NO, COURSE_NO
Table 2: COURSE_NO, COURSE_FEE

Table 1		Table 2	
STUD_NO	COURSE_NO	COURSE_NO	COURSE_FEE
1	C1	C1	1000
2	C2	C2	1500
1	C4	C3	1000
4	C3	C4	2000
4	C1	C5	2000

Third Normal Form (3NF)

We say that a relation schema R is in **3NF** if it is in 2NF and all non-key attributes are **transitively dependent** on the primary key

Third Normal Form (3NF)

We say that a relation schema R is in **3NF** if it is in 2NF and all non-key attributes are not **transitively dependent** on the primary key

Transitive Dependence: A transitive dependence is of the form

$$\begin{aligned} A &\rightarrow B \\ B &\rightarrow C \end{aligned}$$

Therefore, $A \rightarrow C$

Third Normal Form (3NF)

We say that a relation schema R is in **3NF** if it is in 2NF and all non-key attributes are **transitively dependent** on the primary key

Transitive Dependence: A transitive dependence is of the form

$$\begin{aligned} A &\rightarrow B \\ B &\rightarrow C \end{aligned}$$

Therefore, $A \rightarrow C$

Converting a relation in 3NF is like asking whether there is a non-key attribute that whenever changed would require changing another non-key attribute

Third Normal Form (3NF)

Given the table

Supplies(SupplierNo, PartNo, City, County, Quantity)

<u>SupplierNo</u>	<u>PartNo</u>	City	County	Quantity
Acme	Nail3982	Derby	Derbyshire	200
Acme	Screw2309	Derby	Derbyshire	100
Filbert	Bolt4093	Leicester	Leicestershire	125
Filbert	Screw2309	Leicester	Leicestershire	50

Third Normal Form (3NF)

Given the table

Supplies(SupplierNo, PartNo, City, County, Quantity)

Given the following functional dependencies:

SupplierNo, PartNo $\rightarrow$ *Quantity*

SupplierNo $\rightarrow$ *County*

SupplierNo $\rightarrow$ *City*

City $\rightarrow$ *County*

<u>SupplierNo</u>	<u>PartNo</u>	City	County	Quantity
Acme	Nail3982	Derby	Derbyshire	200
Acme	Screw2309	Derby	Derbyshire	100
Filbert	Bolt4093	Leicester	Leicestershire	125
Filbert	Screw2309	Leicester	Leicestershire	50

Third Normal Form (3NF)

County and City are partially functionally dependent on the primary key and therefore the relation is not in 2NF. How do we normalise it?

Supplies(SupplierNo, PartNo, City, County, Quantity)

<u>SupplierNo</u>	<u>PartNo</u>	City	County	Quantity
Acme	Nail3982	Derby	Derbyshire	200
Acme	Screw2309	Derby	Derbyshire	100
Filbert	Bolt4093	Leicester	Leicestershire	125
Filbert	Screw2309	Leicester	Leicestershire	50

Third Normal Form (3NF)

County and City are partially functionally dependent on the primary key and therefore the relation is not in 2NF. How do we normalise it?

Suppliers(SupplierNo, City, County)

Supplies(SupplierNo fk, PartNo, Quantity)

<u>SupplierNo</u>	City	County
Acme	Derby	Derbyshire
Filbert	Leicester	Leicestershire

<u>SupplierNo</u>	<u>PartNo</u>	Quantity
Acme	Nail3982	200
Acme	Screw2309	100
Filbert	Bolt4093	125
Filbert	Screw2309	50

Third Normal Form (3NF)

Now, *Supplies* is in 3NF as Quantity is fully dependent on the primary key and there are no transitive dependencies. *Suppliers* is not in 3NF as

SupplierNo $\rightarrow$ *City*

City $\rightarrow$ *County*

Therefore, *SupplierNo* $\rightarrow$ *County*

<u>SupplierNo</u>	City	County
Acme	Derby	Derbyshire
Filbert	Leicester	Leicestershire

<u>SupplierNo</u>	<u>PartNo</u>	Quantity
Acme	Nail3982	200
Acme	Screw2309	100
Filbert	Bolt4093	125
Filbert	Screw2309	50

Third Normal Form (3NF)

Therefore, we need to remove the transitive dependency
SupplierNo → *City* → *County*

<u>SupplierNo</u>	City	County
Acme	Derby	Derbyshire
Filbert	Leicester	Leicestershire

<u>SupplierNo</u>	<u>PartNo</u>	Quantity
Acme	Nail3982	200
Acme	Screw2309	100
Filbert	Bolt4093	125
Filbert	Screw2309	50

Third Normal Form (3NF)

Therefore, we need to remove the transitive dependency
 $SupplierNo \rightarrow City \rightarrow County$

Suppliers(SupplierNo, City fk)

Cities(City, County)

Supplies(SupplierNo, PartNo, Quantity)

<u>SupplierNo</u>	City fk
Acme	Derby
Filbert	Leicester

<u>City</u>	County
Derby	Derbyshire
Leicester	Leicestershire

<u>SupplierNo</u>	<u>PartNo</u>	Quantity
Acme	Nail3982	200
Acme	Screw2309	100
Filbert	Bolt4093	125
Filbert	Screw2309	50

Third Normal Form (3NF)

Consideration: sometimes, we may need to create an additional table if the number of suppliers is small

Otherwise, a slightly de-normalised schema with no *City* table may be a better choice

<u>SupplierNo</u>	City fk
Acme	Derby
Filbert	Leicester

<u>City</u>	County
Derby	Derbyshire
Leicester	Leicestershire

<u>SupplierNo</u>	<u>PartNo</u>	Quantity
Acme	Nail3982	200
Acme	Screw2309	100
Filbert	Bolt4093	125
Filbert	Screw2309	50

Overview

- Recap: Conceptual & Logical E/R Diagram
- Physical E/R Diagram
- Normal Forms: 1st, 2nd and 3rd
- **Data Redundancy and Update Anomalies**

Anomalies

We want to ensure that each table contains information about one, and only one, kind of entity

Anomalies

We want to ensure that each table contains information about one, and only one, kind of entity

Consider the following table:

BranchEmp(StaffNo, SName, SAddress, Position, Salary, BranchNo, BAddress, TelNo)

Anomalies

We want to ensure that each table contains information about one, and only one, kind of entity

Consider the following table:

BranchEmp(StaffNo, SName, SAddress, Position, Salary, BranchNo, BAddress, TelNo)

- Insertion Anomaly 1: to insert details on a new member of staff, we must include details of the branch
- Insertion Anomaly 2: to insert details about a new branch, we must include data about staff

Anomalies

Consider the following table:

BranchEmp(StaffNo, SName, SAddress, Position, Salary, BranchNo, BAddress, TelNo)

Anomalies

Consider the following table:

BranchEmp(StaffNo, SName, SAddress, Position, Salary, BranchNo, BAddress, TelNo)

- Deletion Anomalies: by deleting a row of the last member of staff at a given branch, we lose all information about that branch
- Modification Anomalies: if we want to change the value of the TelNo of a branch, we need to update all the rows associated with that branch (otherwise inconsistency)

Anomalies

Consider the following table:

BranchEmp(StaffNo, SName, SAddress, Position, Salary, BranchNo, BAddress, TelNo)

- Deletion Anomalies: by deleting a row of the last member of staff at a given branch, we lose all information about that branch
- Modification Anomalies: if we want to change the value of the TelNo of a branch, we need to update all the rows associated with that branch (otherwise inconsistency)

Therefore, we solve this problem through normalization:

Employees(StaffNo, SName, SAddress, Position, Salary, BranchNo fk)
Branch(BranchNo, BAddress, TelNo)

Aims of the Session

You should now be able to:

1. Understand the main concepts behind different types of E/R diagrams
2. Explain the steps to produce a normalised database
3. Describe the different types of anomalies in the creation of a relational database

References

Silberschatz, A., Korth, H. F., and Sudarshan, S. (2010), *Database System Concepts*, 6th Edition. McGraw-Hill.

- Chapter 8 – Relational Database Design (Sections 8.1, 8.2, 8.3 and 8.8)